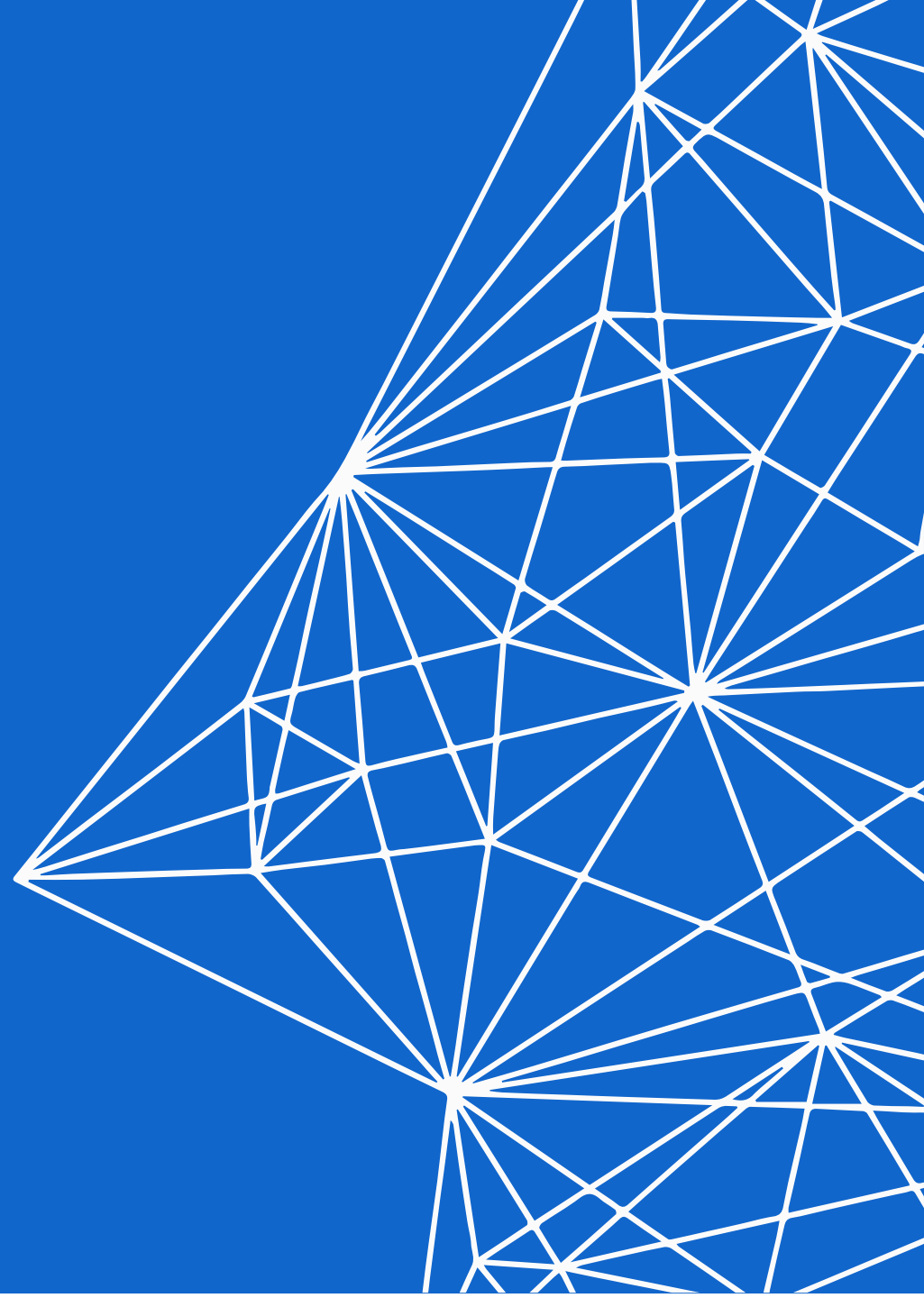




第三节：图像处理基础

杨聪





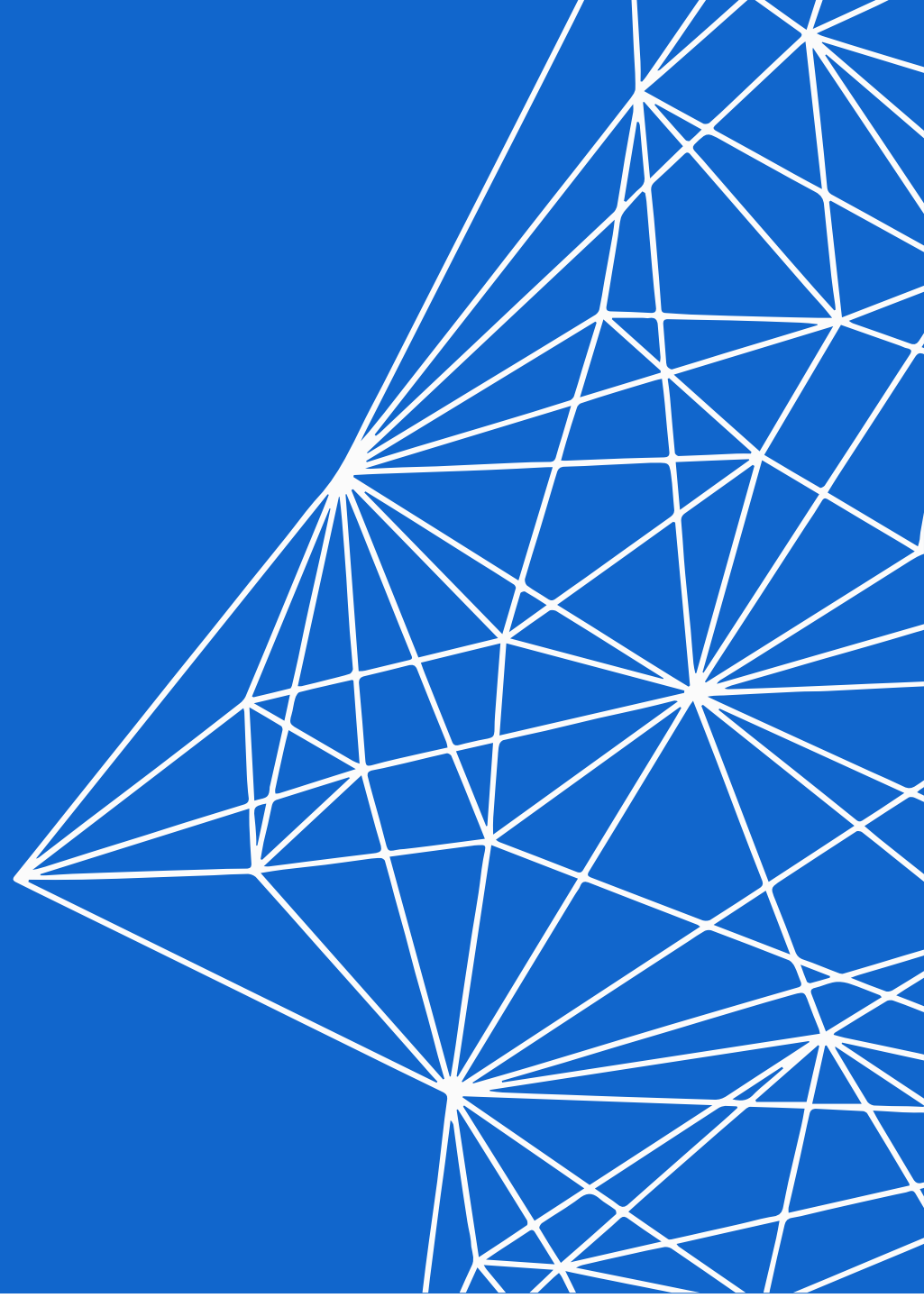
目录

CONTENTS

1. 计算机视觉概述

2. 图像处理基础

- 图像的数值表示
- 图像二值化：图像分割
- 图像的量化
- 图像中的几何操作
- 图像边缘检测





RGB: 常见的三原色

RGB的局限性

RGB 颜色空间的三个分量都与亮度密切相关，即只要亮度改变，三个分量都会随之相应地改变，而没有一种更直观的方式来表达。

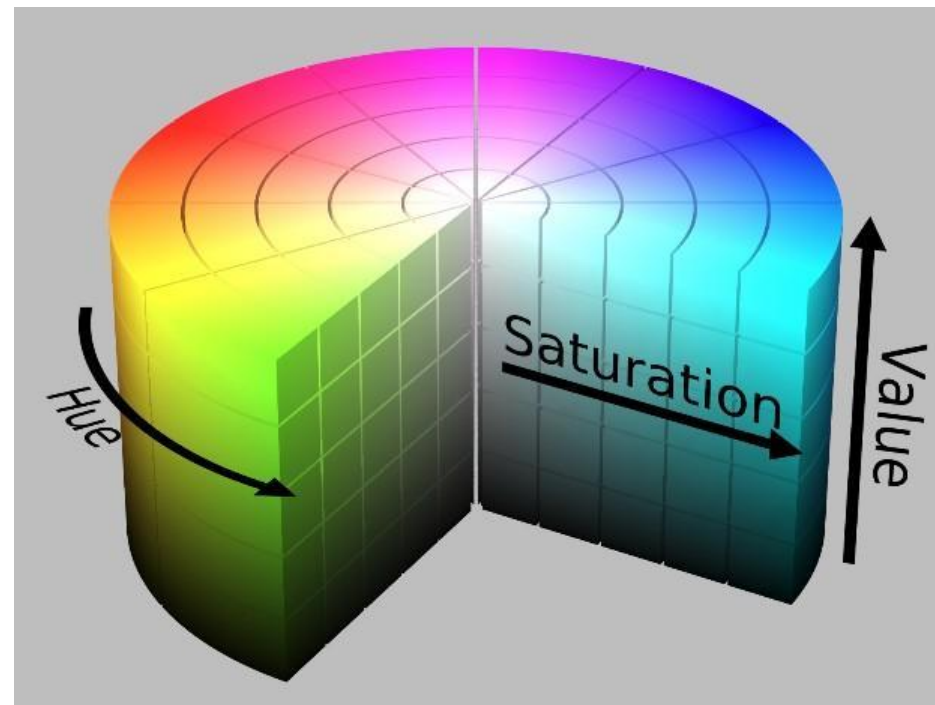
人眼对于这三种颜色分量的敏感程度是不一样的，在单色中，人眼对红色最不敏感，蓝色最敏感，所以 RGB 颜色空间是一种均匀性较差的颜色空间。

如果颜色的相似性直接用欧氏距离来度量，其结果与人眼视觉会有较大的偏差。对于某一种颜色，我们很难推测出较为精确的三个分量数值来表示。

HSV：将颜色表示为色调（Hue）、饱和度（Saturation）及明度（Value）的方式。可以更好地描述颜色的感觉特征。

用这个圆柱体来表示 HSV 颜色空间，圆柱体的横截面可以看做是一个极坐标系：

- H 用极坐标的极角表示
- S 用极坐标的极轴长度表示
- V 用圆柱中轴的高度表示

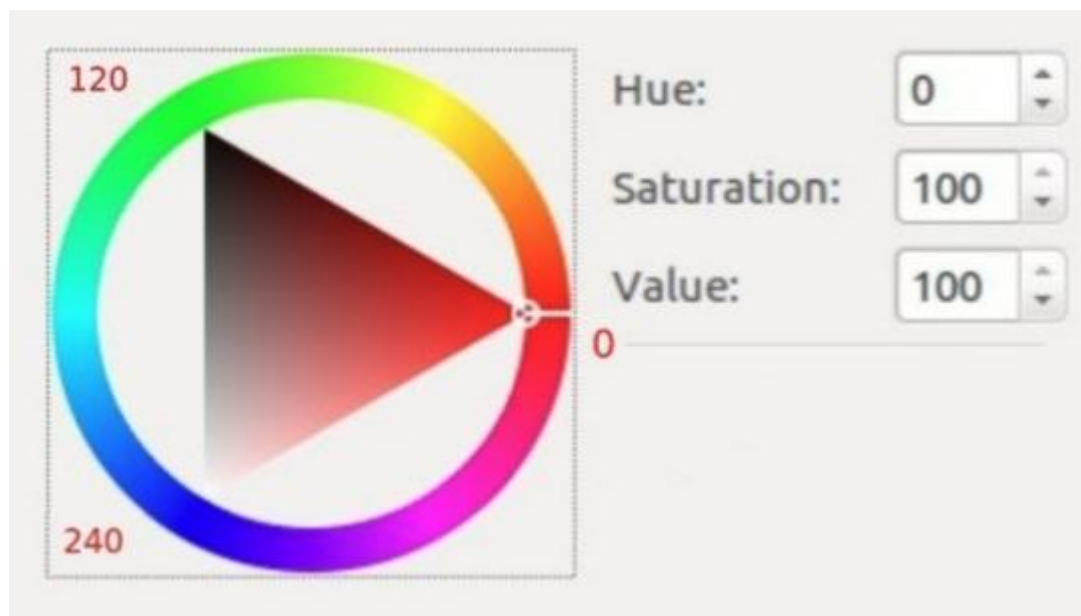


HSV：将颜色表示为色调（Hue）、饱和度（Saturation）及明度（Value）的方式。可以更好地描述颜色的感觉特征。

Hue 用角度度量，取值范围为 $0 \sim 360^\circ$ ，表示色彩信息，即所处的光谱颜色的位置，表示如右。

颜色圆环上所有的颜色都是光谱上的颜色，从红色开始按逆时针方向旋转，Hue=0 表示红色，Hue=120 表示绿色，Hue=240 表示蓝色等等。

(看Fireworks的demo)

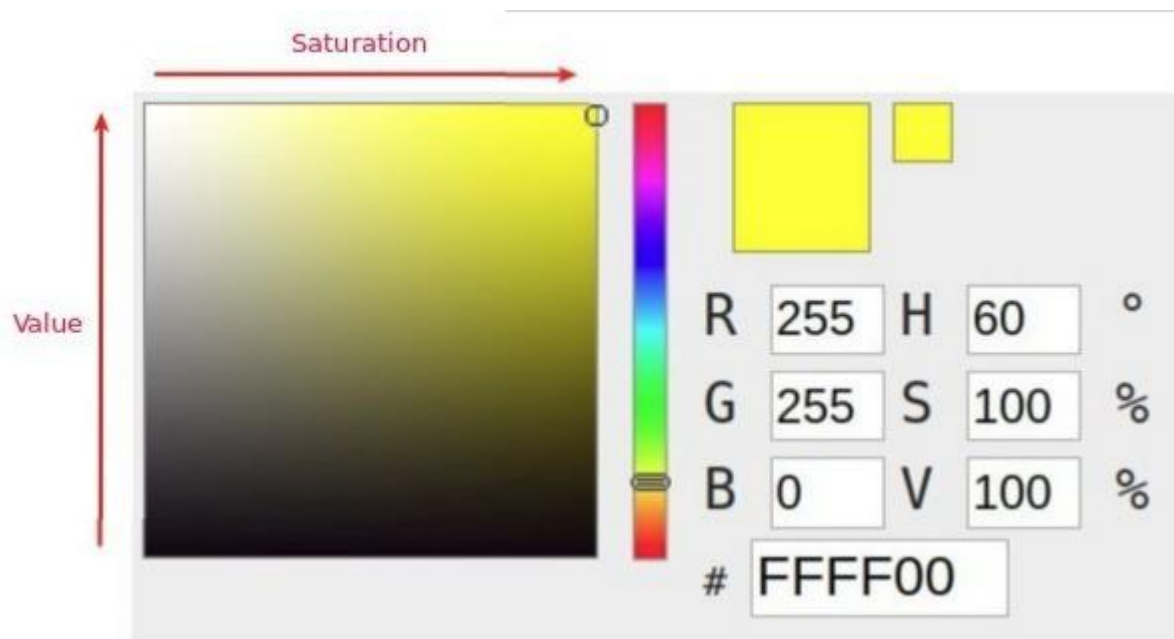


HSV: 将颜色表示为色调 (**Hue**)、饱和度 (**Saturation**) 及明度 (**Value**) 的方式。可以更好地描述颜色的感觉特征。

在 GRB 中 颜色由三个值共同决定, 比如黄色为即 (255,255,0); 在 HSV 中, 黄色只由一个值决定, Hue=60 即可。

水平方向表示饱和度, 饱和度表示颜色接近光谱色的程度。饱和度越高, 说明颜色越深, 越接近光谱色饱和度越低, 说明颜色越浅, 越接近白色。饱和度为0表示纯白色。取值范围为0 ~ 100%, 值越大, 颜色越饱和。

竖直方向表示明度, 决定颜色空间中颜色的明暗程度, 明度越高, 表示颜色越明亮, 范围是 0-100%。明度为0表示纯黑色 (此时颜色最暗)。





可以通俗理解为：

在Hue一定的情况下，饱和度减小，就是往光谱色中添加白色，光谱色所占的比例也在减小，饱和度减为0，表示光谱色所占的比例为零，导致整个颜色呈现白色。明度减小，就是往光谱色中添加黑色，光谱色所占的比例也在减小，明度减为0，表示光谱色所占的比例为零，导致整个颜色呈现黑色。

HSV 对用户来说是一种比较直观的颜色模型。我们可以很轻松地得到单一颜色，即指定颜色角H，并让 $V=S=1$ ，然后通过向其中加入黑色和白色来得到我们需要的颜色。增加黑色可以减小V而S不变，同样增加白色可以减小S而V不变。例如，要得到深蓝色， $V=0.4$ $S=1$ $H=240$ 度。要得到浅蓝色， $V=1$ $S=0.4$ $H=240$ 度。

HSV 的拉伸对比度增强就是对 S 和 V 两个分量进行归一化(min-max normalize)即可，H 保持不变。

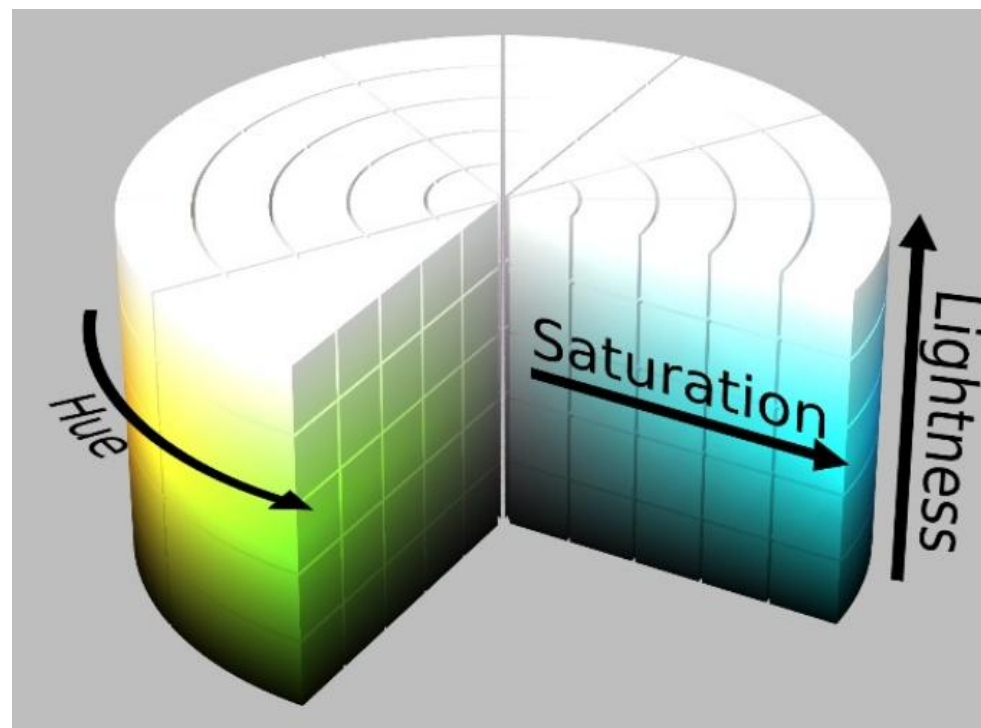
RGB颜色空间更加面向于工业，而HSV更加面向于用户，大多数做图像识别这一块的都会运用HSV颜色空间，因为HSV颜色空间表达起来更加直观！

HLS: 和 HSV 比较类似, 也有三个分量, Hue (色调)、Saturation (饱和度)、Lightness (亮度)。

HLS 和 HSV 的区别就是最后一个分量不同, HLS 的是 Light(亮度), HSV 的是 Value(明度)。

- HLS 中的 L 分量为亮度, 亮度为100, 表示白色, 亮度为0, 表示黑色;
- HSV 中的 V 分量为明度, 明度为100, 表示光谱色, 明度为0, 表示黑色。

提取白色物体时, 使用 HLS 更方便, 因为 HSV 中的Hue里没有白色, 白色需要由S和V共同决定 ($S=0, V=100$)。而在 HLS 中, 白色仅由亮度L一个分量决定。所以检测白色时使用 HSL 颜色空间更准确。





YUV: YUV是编译true-color颜色空间 (color space) 的种类, Y'UV, YUV, YCbCr, YPbPr等专有名词都可以称为YUV, 彼此有重叠。“Y”表示明亮度 (Luminance或Luma), 也就是灰阶值, “U”和“V”表示的则是色度 (Chrominance或Chroma), 作用是描述影像色彩及饱和度, 用于指定像素的颜色。

Y'UV, YUV, YCbCr, YPbPr所指涉的范围, 常有混淆或重叠的情况。从历史的演变来说, 其中YUV和Y'UV通常用来编码电视的模拟信号, 而YCbCr则是用来描述数字的视频信号, 适合视频与图片压缩以及传输, 例如MPEG、JPEG。

Y'UV的发明是由于彩色电视与黑白电视的过渡时期。黑白视频只有Y (Luma, Luminance) 视频, 也就是灰阶值。到了彩色电视规格的制定, 是以YUV/YIQ的格式来处理彩色电视图像, 把UV视作表示彩度的C (Chrominance或Chroma), 如果忽略C信号, 那么剩下的Y (Luma) 信号就跟之前的黑白电视频号相同, 这样一来便解决彩色电视机与黑白电视机的兼容问题。Y'UV最大的优点在于只需占用极少的带宽。

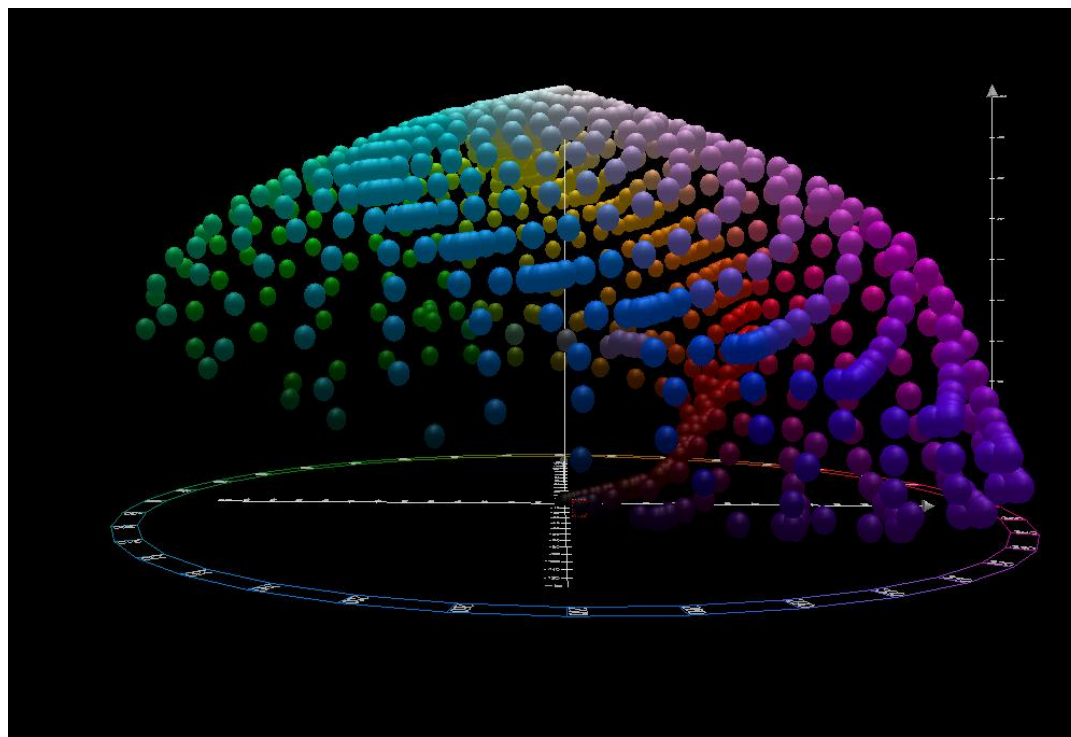
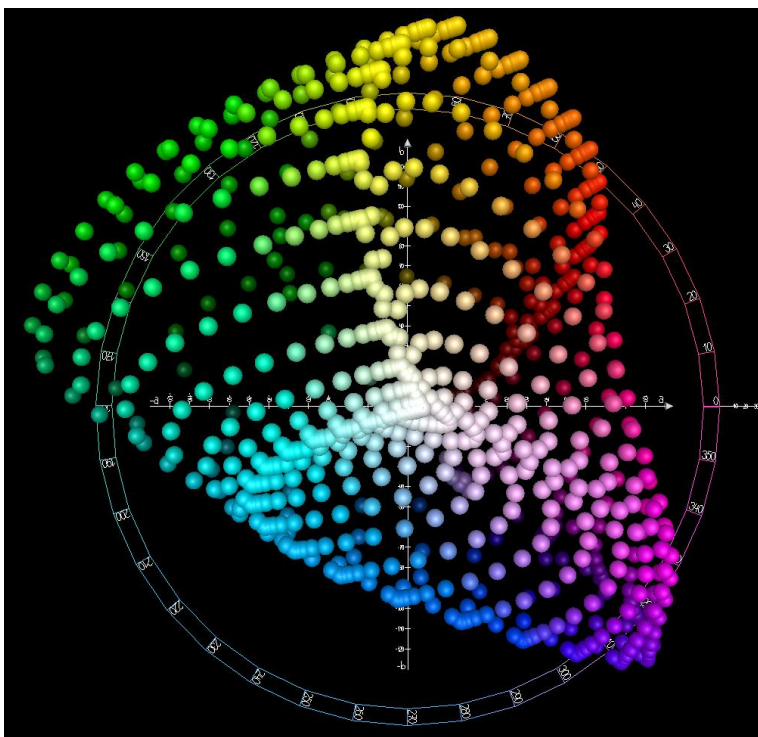
总结：

YUV是一种模拟信号，其色彩模型源于RGB模型，即亮度与色度分离，适合图像算法的处理，常应用于在模拟广播电视中，其中 $Y \in [0,1]$ $U,V \in [-0.5,0.5]$ 。

YCbCr是一种数字信号，其色彩模型源于YUV模型，它是YUV压缩和偏移的版本（所谓偏移就是从 $[-0.5,0.5]$ 偏移到 $[0,1]$ ，因此计算时候会加128），在数字视频领域应用广泛，是计算机中应用最多的格式，包括JPEG，MPEG，H.264/5，AVS等都采用YCbCr格式，我们通常广义的讲的YUV，严格的讲应该就是YCbCr。

CIE L*a*b: 是一种国际通用的色彩表示标准。有鉴于色彩标准过多, 无法统一, 国际标准色彩协会色彩特别在1931年制订了一套国际通用的色彩表示标准。这套色彩系统与印刷设备与器材都没有任何关系。

L*a*b由一个垂直轴心表示明亮度 (Luminance) 的全白到全黑, 两个水平延伸面表示色彩, 其中之一为红到绿色, 另外一个面表示蓝到黄色。 (<https://zhuanlan.zhihu.com/p/58395048>)



RGB



R
(G=0,
B=0)

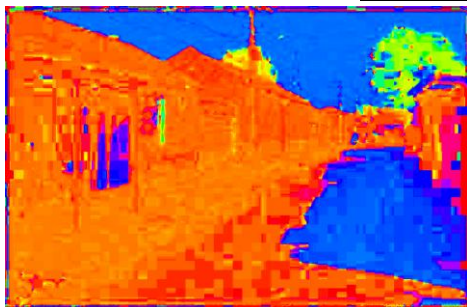


G
(R=0,
B=0)



B
(R=0,
G=0)

HSV



H
(S=1,
V=1)



S
(H=1,
V=1)



V
(H=1,
S=0)



YCbCr



Y
(Cb=0.5,
Cr=0.5)



Cb
(Y=0.5,
Cr=0.5)



Cr
(Y=0.5,
Cb=0.5)

CIE L*a*b*



L
(a=0,
b=0)



a
(L=65,
b=0)



b
(L=65,
a=0)

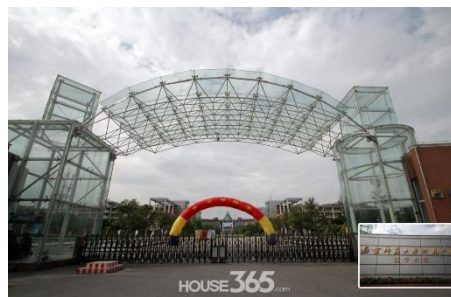


➤ 各种颜色空间及描述

Tab. 1 Various color spaces and their brief descriptions (six categories)

Color spaces	Descriptions
RGB	Red, green, blue
YUV, YIQ, YPbPr, YCbCr, JPEG-YCbCr, YDbDr	Luminance, chrominance
HSV, HSL, HSI	Hue, saturation, value/lightness/intensity
$I_1 I_2 I_3$	Linear transform of RGB
CIE XYZ, CIE LUV, CIE LCH, CIE LAB, CIE UVW, CAT02 LMS	Tristimuli, chromaticity, and colorimetric systems
Opponent RGB ^[8] Double opponent RGB ^[9]	Opponent theory

RGB彩色图像



$610 \times 950 \times 3$



610×950

Red通道



610×950

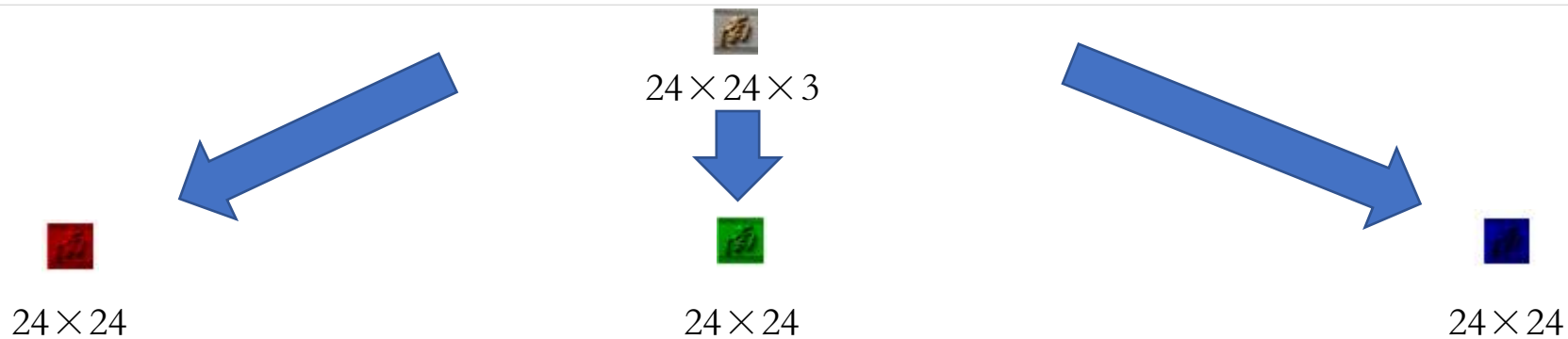
Green通道



610×950

Blue通道

RGB彩色图像



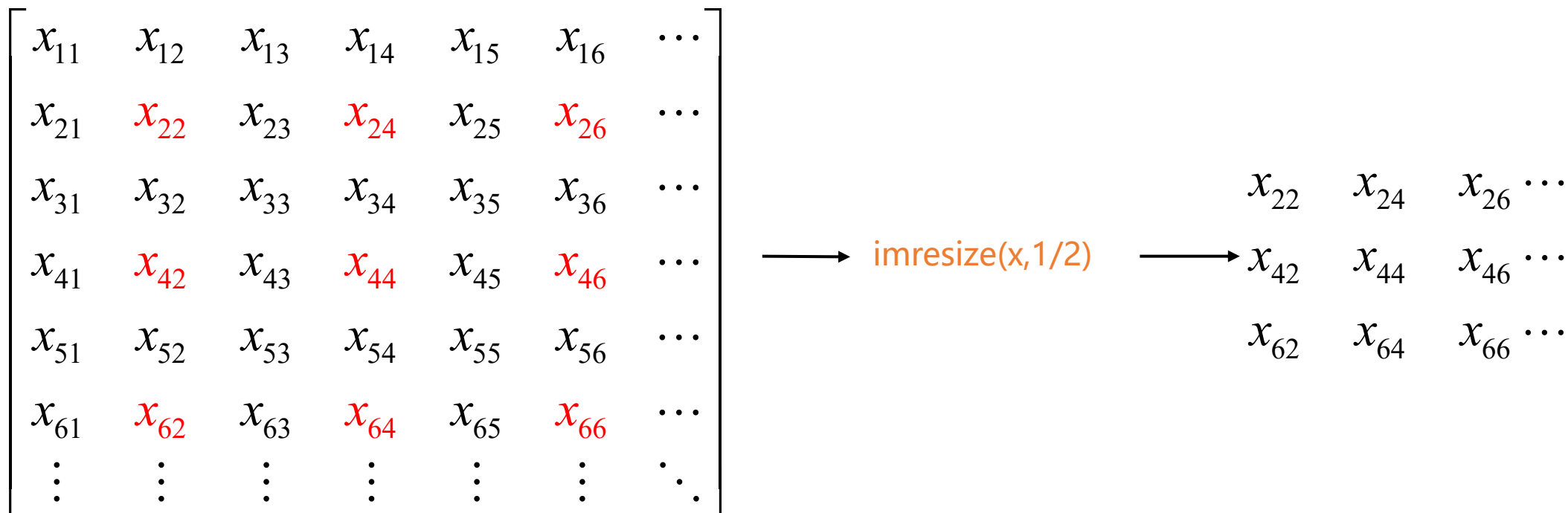
255	147	134	146	137	145	147	145	157	151	152	162	144	161	151	160	157	166	155	162	160	153	162	162
231	122	93	93	93	99	91	92	98	101	112	106	110	120	102	112	120	131	122	132	130	139	133	131
255	122	102	113	110	115	114	108	103	107	116	105	123	111	106	100	116	121	121	122	119	119	115	116
250	150	142	135	128	129	129	128	130	118	119	118	118	109	123	124	144	120	113	128	130	132	132	129
253	152	157	159	156	149	150	158	159	160	157	205	186	148	164	164	154	103	85	153	162	165	156	159
254	145	151	145	146	154	154	135	143	168	179	197	190	132	103	73	46	37	61	137	153	159	159	159
255	153	154	146	152	146	145	151	150	152	161	134	128	100	32	24	27	64	108	126	138	155	161	148
249	144	156	145	142	137	149	151	145	116	88	151	129	79	31	91	126	130	134	136	147	161	154	159
252	150	141	154	131	147	158	146	138	154	139	145	87	113	109	85	51	81	144	122	145	152	160	154
246	152	150	127	163	145	121	142	177	189	162	55	73	106	38	27	43	73	165	76	115	150	160	159
245	156	136	142	157	160	128	169	179	132	53	23	104	107	62	52	113	137	127	52	82	132	151	157
249	146	143	148	139	150	99	70	67	92	37	115	143	81	65	78	123	146	93	49	88	128	143	159
247	147	143	153	128	135	32	13	59	159	134	135	71	21	43	63	124	139	90	51	91	136	149	143
243	148	143	148	148	106	37	77	107	139	144	143	97	124	85	93	166	113	44	70	99	132	155	155
241	152	145	140	150	90	54	87	78	88	137	77	79	102	55	86	137	73	37	80	105	137	150	146
249	146	140	137	133	93	54	85	127	130	113	78	46	28	59	152	90	51	37	100	118	143	146	148
242	143	146	149	125	68	40	89	125	110	90	62	40	55	86	68	32	42	76	111	126	140	151	149
248	134	136	121	112	63	53	84	88	56	30	19	47	82	100	77	44	68	95	117	133	145	151	151
249	147	143	133	109	55	41	90	110	88	57	51	74	115	124	122	118	116	124	135	141	150	154	150
251	135	123	134	92	54	44	88	131	130	124	104	109	114	133	144	159	140	134	139	140	147	151	145
239	112	82	81	87	69	52	65	70	85	84	93	81	84	90	96	92	97	96	95	102	108	107	107
244	113	96	97	109	109	105	92	92	96	93	99	104	99	111	105	92	97	101	102	103	103	103	105
246	134	121	119	117	113	111	115	122	124	111	116	112	117	117	120	119	109	107	109	103	101	104	104
242	134	136	138	139	137	143	137	138	142	141	145	149	150	140	144	145	149	143	138	144	146	144	146

对图片来说，计算机看到的都是数字（像素值）！

图像分辨率

- 图像的分辨率是图像中像素的物理大小。
- 图像分辨率= X方向的像素数 × Y方向的像素数
 - 分辨率越高，用于显示图像的像素就越多。

Matlab **imresize** function



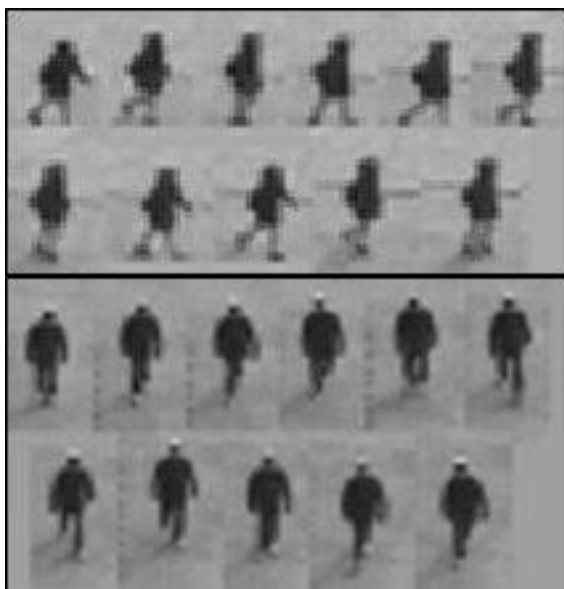
图像二值化 (图像分割)

- 图像分割：将图像分成不相交的有物理意义的区域集合。

➤ 手语识别

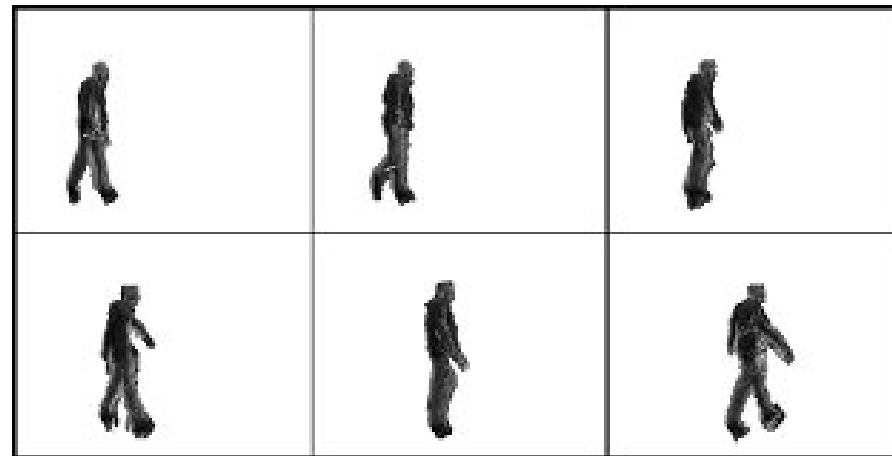


➤ 人类动作跟踪



视频

分割



分析和识别

图像二值化（图像分割）的阈值

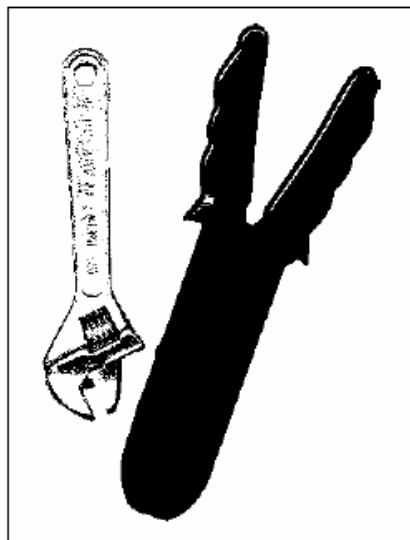
- 灰度图像的二值化处理变换函数

$$g(x, y) = \begin{cases} 0, & f(x, y) < T \\ 1, & f(x, y) \geq T \end{cases}$$

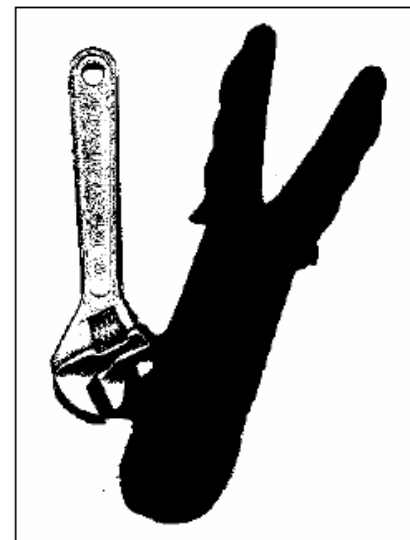
阈值(Threshold)

黑色像素值为1,
白色像素值为0

Original

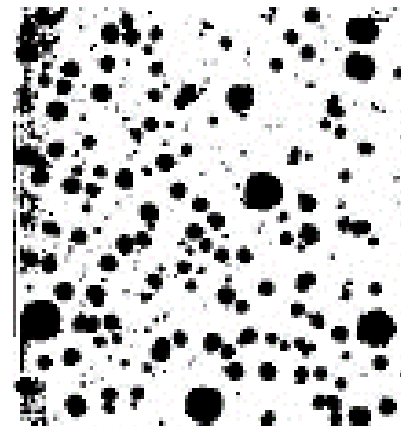
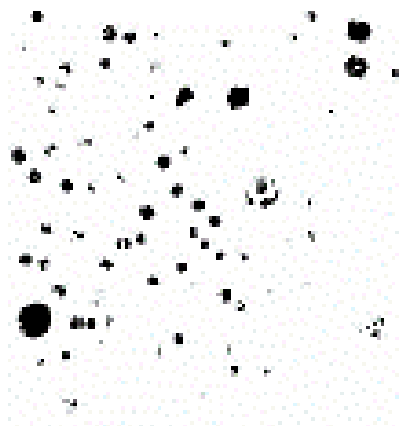
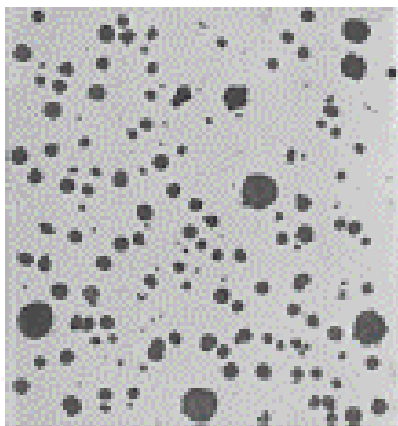


Threshold = 50



Threshold = 75

图像二值化（图像分割）的阈值: 固定阈值



黑色：像素值为1

n: Original image n1: Threshold too low n2: Threshold too high

```
n=imread('nodules1.jpg');  
figure(1); imshow(n);  
n1=im2bw(n,0.35);  
n2=im2bw(n,0.75);  
figure(2), imshow(n1);  
figure(3), imshow(n2);
```

图像二值化（图像分割）的阈值: 自动二值化阈值算法-循环阈值选择方法 (Iterative threshold selection)

- ❖ 以近似阈值开始，然后再调整这个阈值估计。
- ❖ 期望通过阈值化得到子图像的某些性质可以选择新阈值，以得到比前一个阈值更好的分割结果。

例: Matlab function *opthr*

黑色像素值为1,
白色像素值为0

```
im = imread('boy.jpg');  
I = rgb2gray(im);  
I=double(I);  
T=opthr(I);  
bim=(I>T);  
subplot(1,2,1), imshow(I, gray(256));  
subplot(1,2,2), imshow(bim)
```



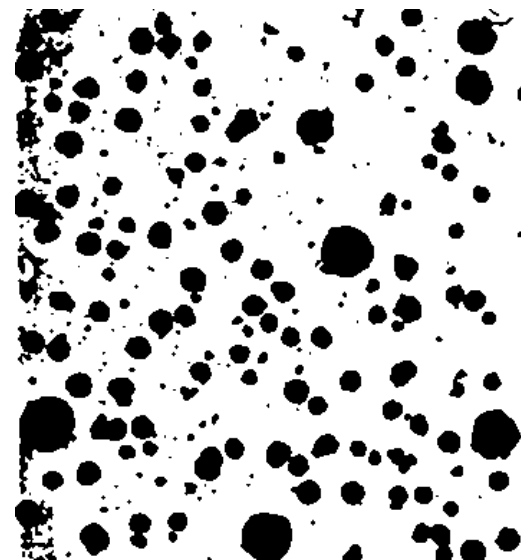
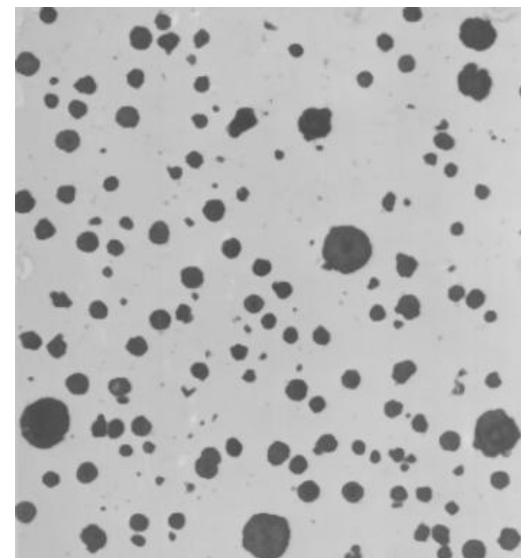
图像二值化 (图像分割) 的阈值: 最优阈值算法-大津法 (Otsu's Method)

- 大津法: 最大化阈值后黑白像素的类间方差

例:

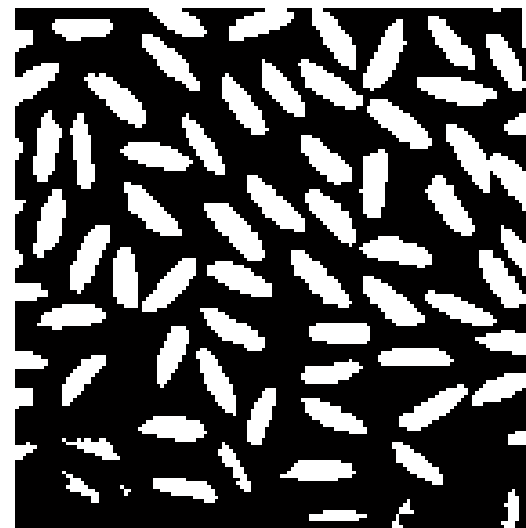
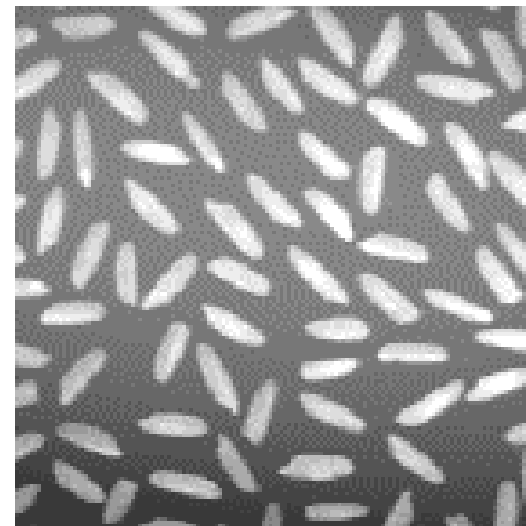
```
function level = graythresh(I)
GRAYTHRESH Compute global threshold
using Otsu's method. Level is a normalized
intensity value that lies in the range [0, 1].
```

```
>>n=imread('nodules1.tif');
>> tn=graythresh(n)
tn = 0.5804
>> imshow(im2bw(n,tn))
```



例:

```
r=imread('rice.tif');  
graythresh;  
subplot(211);  
imshow;  
subplot(212);  
imshow(im2bw(r, tr))
```



- 量化是指使用多大范围的数值来表示图像采样之后的每一个像素点。
 - ✓ 一组 n 个量化级别包含整数0、1、2、... $n-1$ 。
 - ✓ 0和 $n-1$ 通常分别为黑色和白色，中间层呈现不同的灰色。
 - ✓ 量化位数越来越大，表示图像可以拥有更多的颜色，自然可以产生更为细致的图像效果。
- 通常， $n=2^b$ ， b 是用于量化的位数
 - ✓ 如果 $b=8$ ，就有256个可能的灰度级。
 - ✓ 如果 $b=1$ ，则只有两个灰度值：0和1，此图像称为二进制图像（**二值图像**）



Represent an image with n greyscales by dividing the range of greyscales into n equal ranges and mapping the ranges into $0 \sim n-1$

➤ **Example ($n=4$):**

Original values	Output value
0 - 63	0
64 - 127	1
128 - 191	2
192 - 255	3

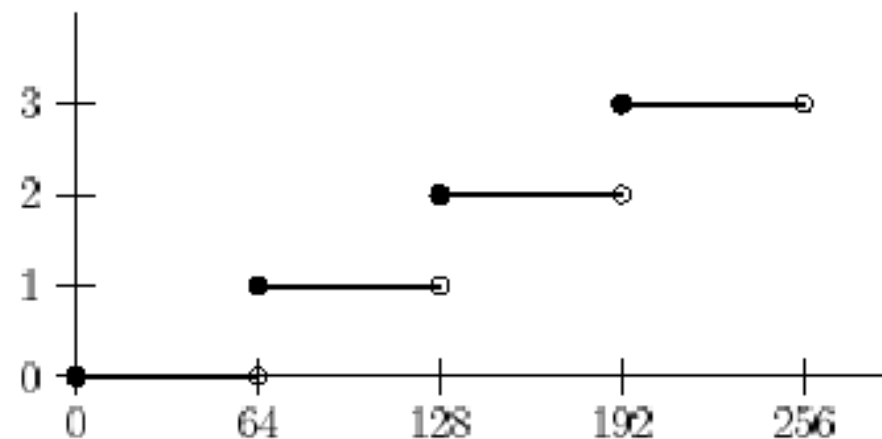


Figure 3.1: A mapping for uniform quantization

A mapping for uniform quantization

例：图像质量随着每个像素的位数减少变差：



#greyscales: 256
(8 bits)

8
(3 bits)

4
(2 bits)

2
(1 bit)

Notice contouring effect.

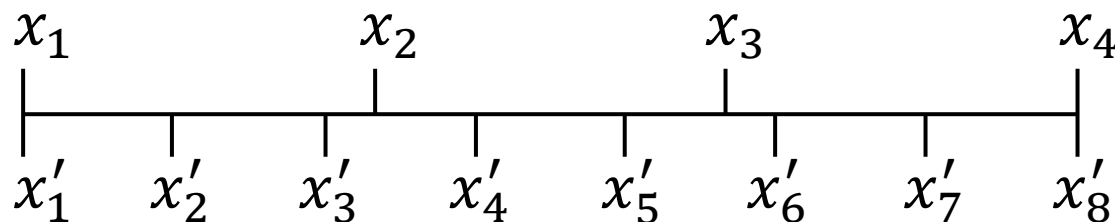
轮廓效应

二值神经网络
神经网络压缩

缩小或缩放图像的大小（或图像的一部分）：

- 改变图像的视觉外观；
- 改变信息量；
- 在多阶段图像处理过程中，作为底层处理。

内插： 一维信号



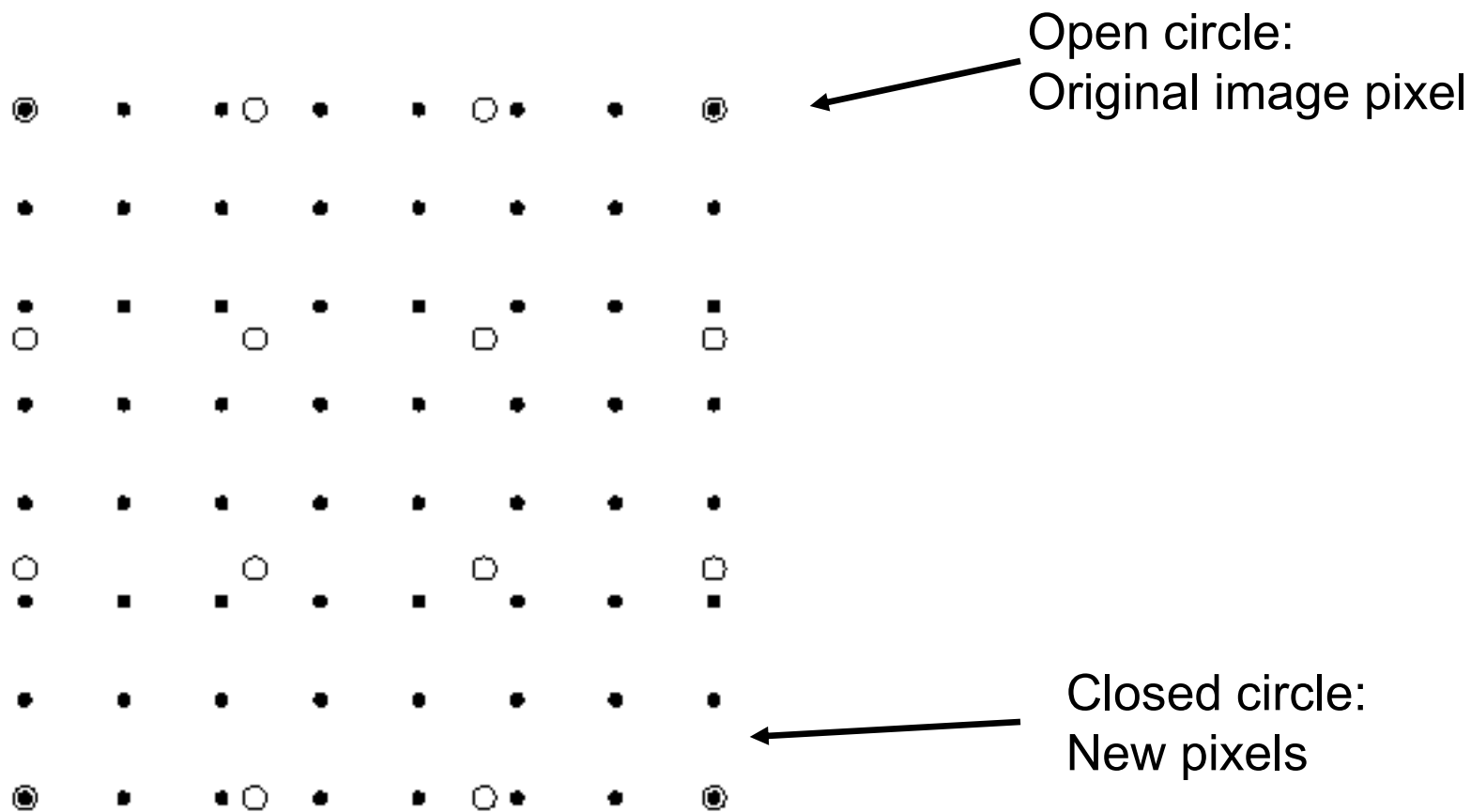
x_1, x_2, x_3, x_4 are original points

Replacing four points with eight

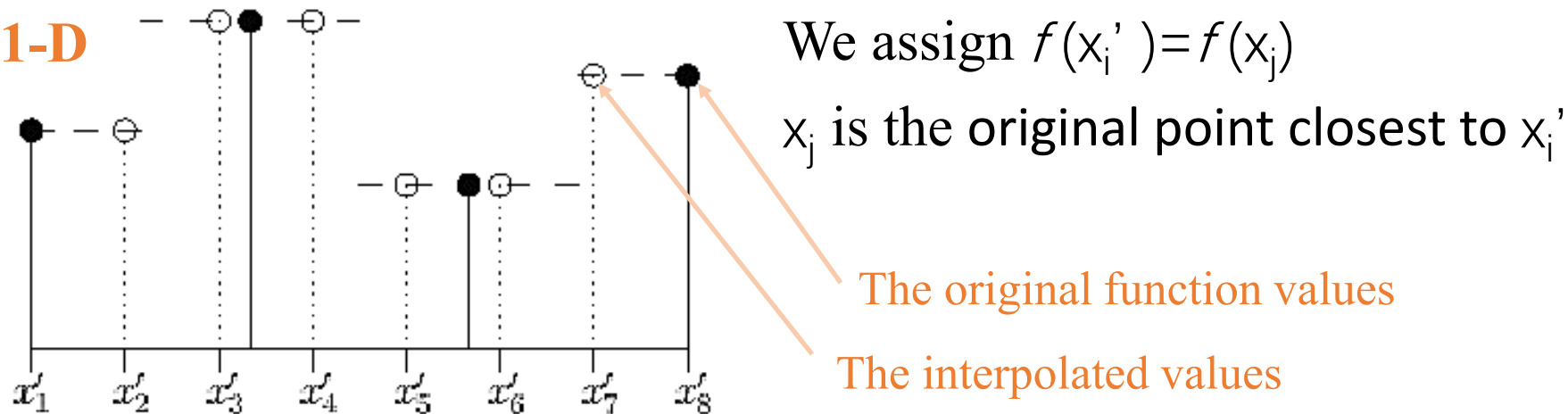
We have to guess at possible function values $f(x'_i)$. This guessing at function values is called **interpolation**.

Interpolation has great significance in general image/video processing

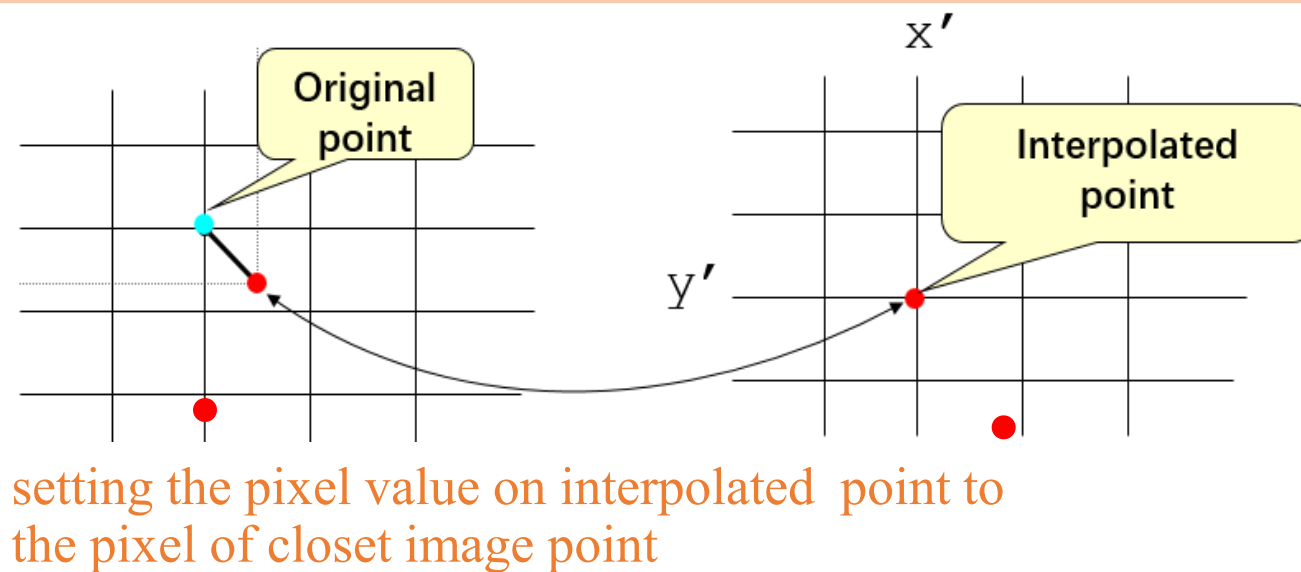
例：图像上的插值 (4x4 $\longrightarrow$ 8x8)



1-D

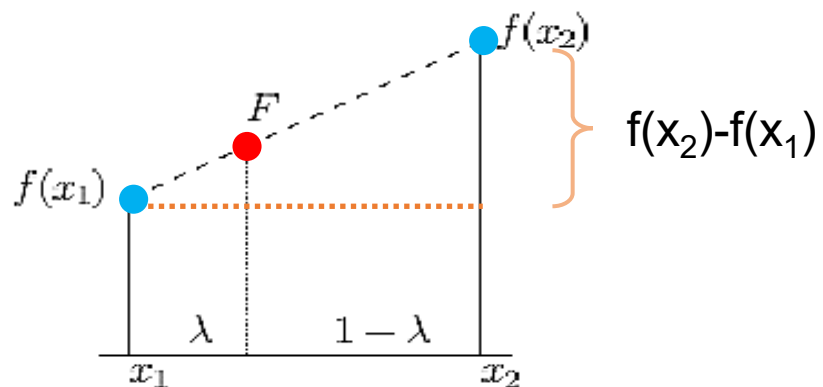


2-D



线性内插

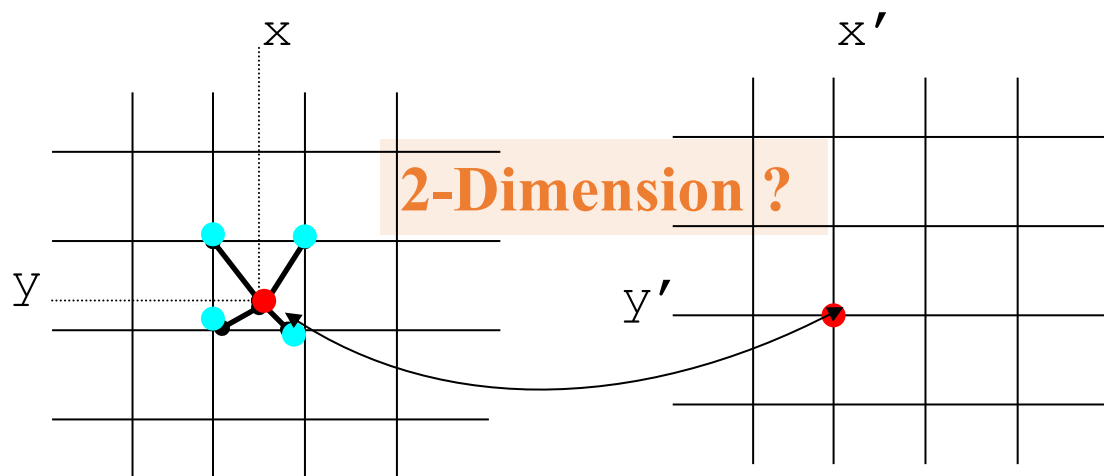
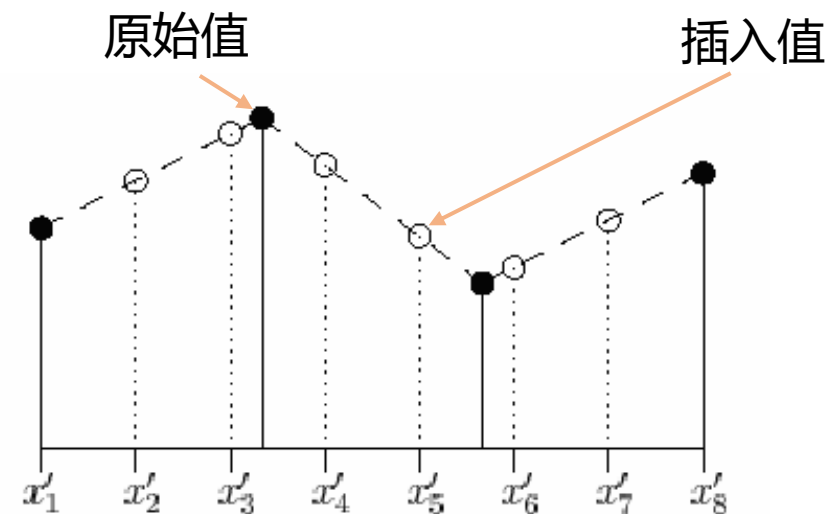
计算插入值:



$$\frac{F - f(x_1)}{\lambda} = \frac{f(x_2) - f(x_1)}{1}$$

$\Rightarrow$

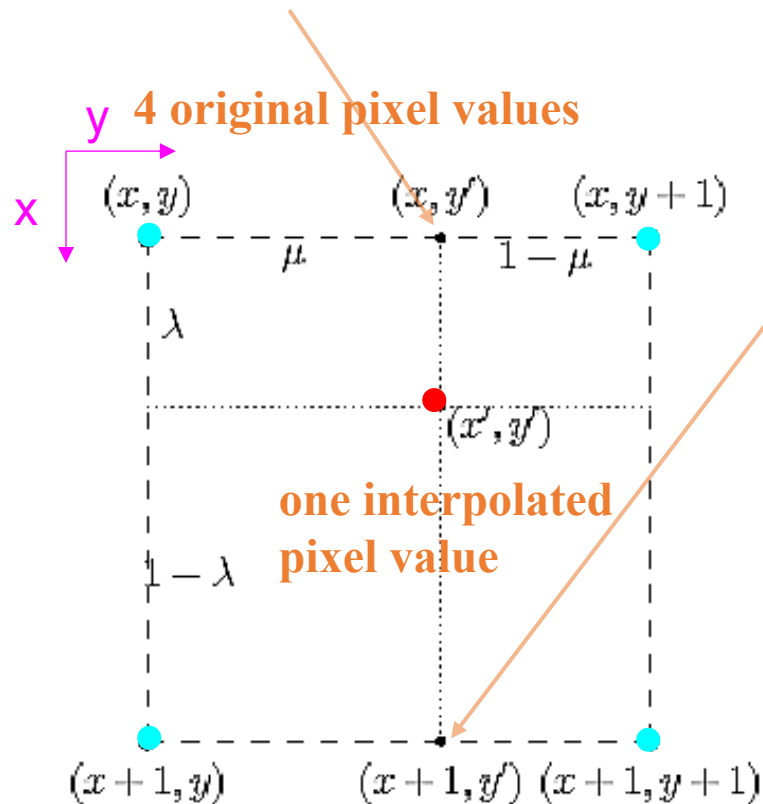
$$F = \lambda f(x_2) + (1 - \lambda) f(x_1)$$



setting the pixel value on interpolated point by using linear interpolation

How a 4x4 image would be interpolated to produce an 8x8 image?

$$f(x, y') = \mu f(x, y + 1) + (1 - \mu) f(x, y)$$



$$f(x + 1, y') = \mu f(x + 1, y + 1) + (1 - \mu) f(x + 1, y)$$

Along the y' column we have

$$f(x', y') = \lambda f(x + 1, y') + (1 - \lambda) f(x, y')$$



$$f(x', y') = \lambda \mu f(x + 1, y + 1) + \lambda (1 - \mu) f(x + 1, y) + (1 - \lambda) \mu f(x, y + 1) + (1 - \lambda) (1 - \mu) f(x, y)$$

Substituting in the values just obtained will result in

$$\begin{aligned} f(x', y') &= \lambda(\mu f(x+1, y+1) + (1-\mu)f(x+1, y)) \\ &\quad + (1-\lambda)(\mu f(x, y+1) + (1-\mu)f(x, y)) \\ &= \lambda\mu f(x+1, y+1) + \lambda(1-\mu)f(x+1, y) \\ &\quad + (1-\lambda)\mu f(x, y+1) + (1-\lambda)(1-\mu)f(x, y) \end{aligned}$$

This is the equation for **bilinear interpolation**

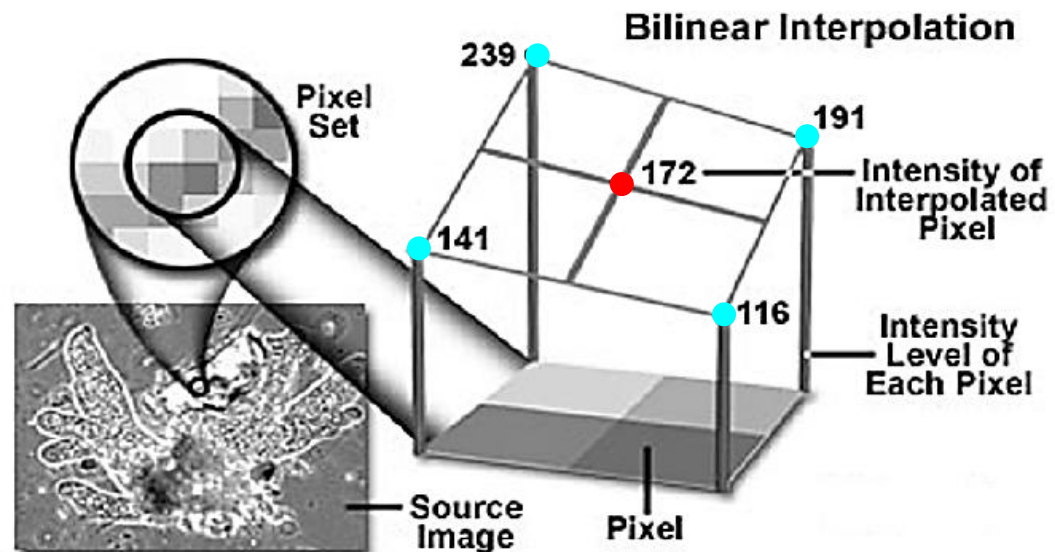


Image Scaling by Matlab resize

```
B = imresize(A, m, method)
B = imresize(A,[mrows ncols], method)
```

 返回一个 mrows 行, ncols 列的图像

imresize resizes an image using the specified interpolation method.
The method is a string that can have one of these values:

- ‘nearest’ (default) uses nearest neighbor interpolation
- ‘bilinear’ uses bilinear interpolation
- ‘bicubic’ uses bicubic interpolation

双三次插值(二元三次)

➤ Example

```
im = imread('cman.tif');
head = im(33:96,90:153);
head1 = imresize(head,3,'nearest');
head2 = imresize(head,3,'bilinear');
```



Image Scaling by Matlab resize



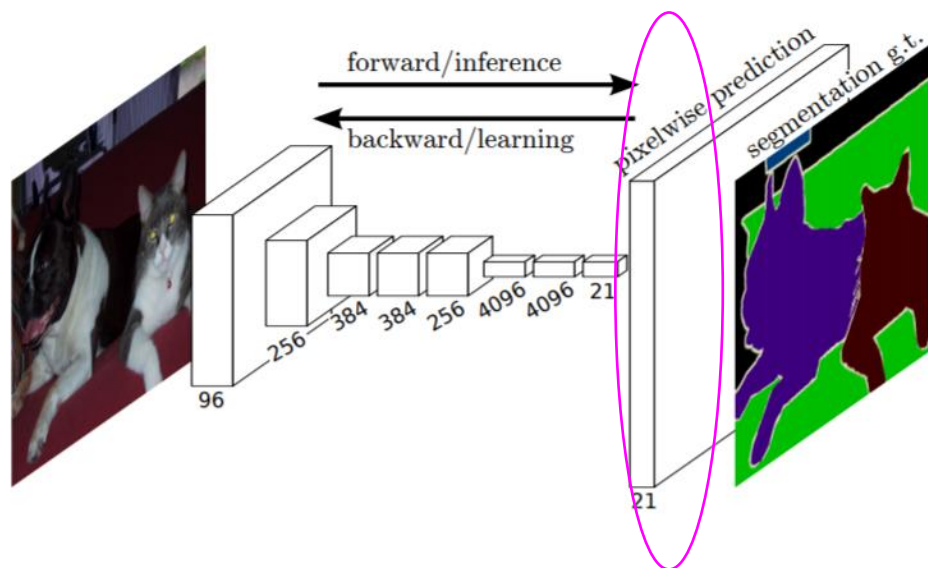
(a) Nearest neighbour scaling



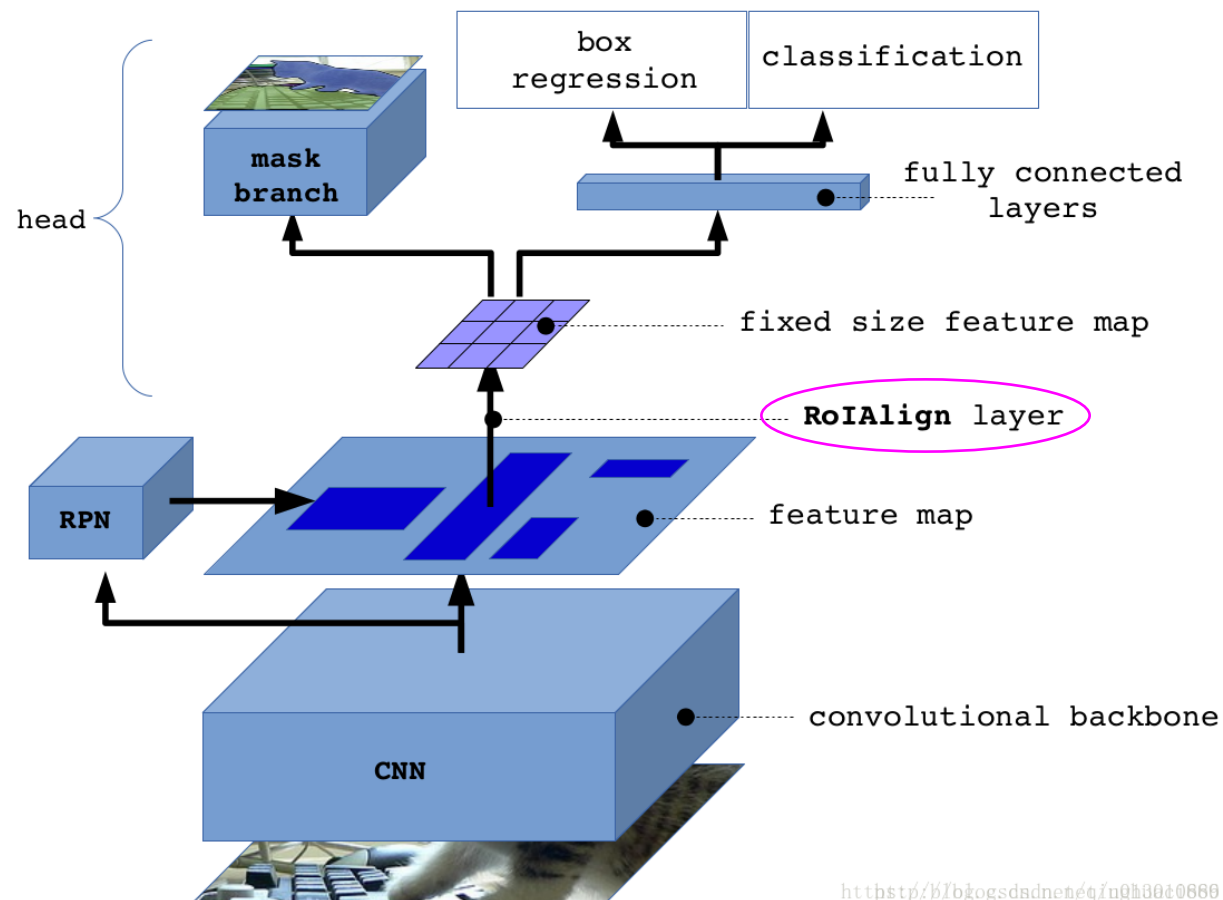
(b) Bilinear interpolation

NN interpolation gives blocky effect and edges appear jagged (锯齿的).
Bilinear interpolation is much smoother

图像插值在深度学习领域的应用举例



Fully Convolutional Network (FCN)



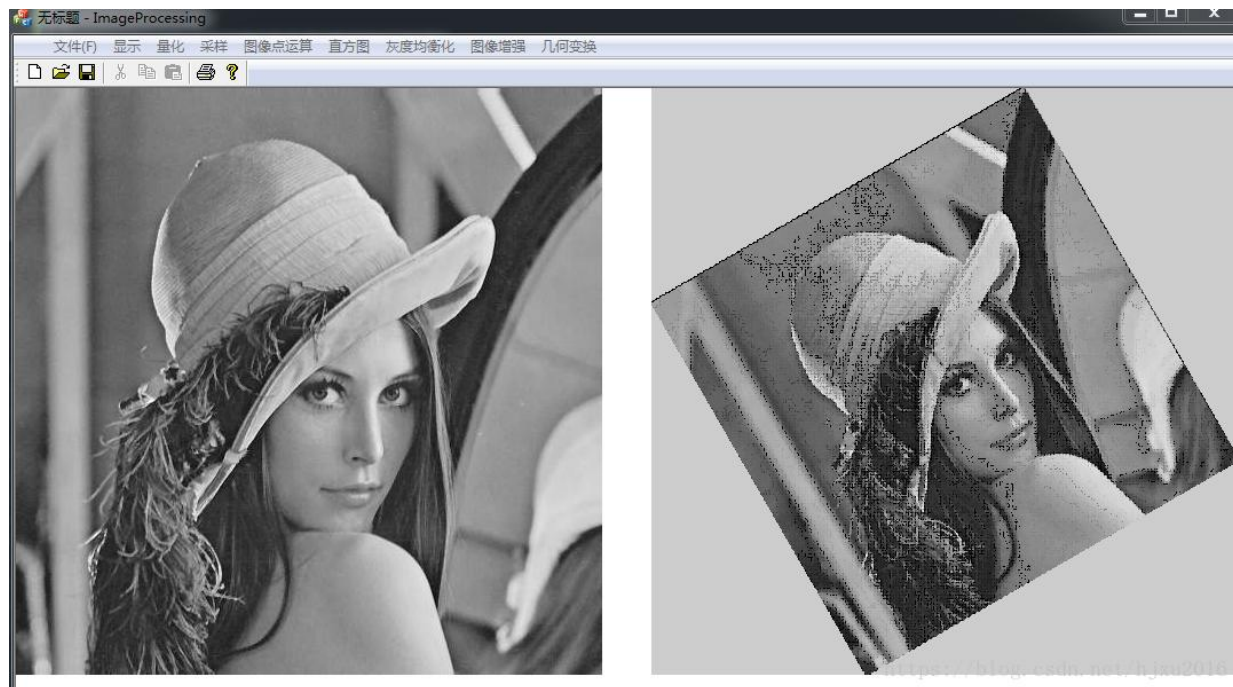
Mask R-CNN

<http://b1bkgosdadnen4q1nq0h00t0660>

图像的旋转变换

旋转是一种几何变换，将输入图像中的一个图像元素的位置映射到输出图像中的一个位置上，方法是使它绕原点旋转一个角度。

- 常见用于改善图像的视觉外观。
- 在各种应用程序中，作为预处理。
- 旋转是仿射变换的一种特殊情况。



中心旋转

➤ Mapping of a point

Mapping of a point (x,y) to another (x',y') through a counter-clockwise rotation of θ

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



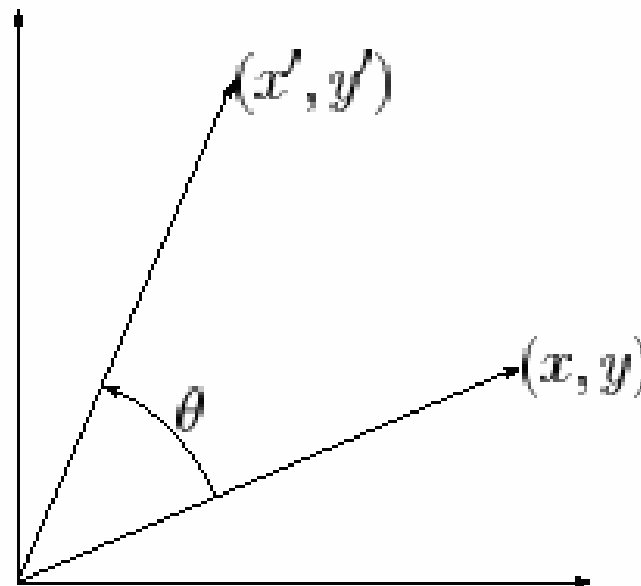
$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix} \begin{bmatrix} x' \\ y' \end{bmatrix}$$

计算通常需要4次乘法和2次加法。

思考：怎样通过3次乘法和3次加法实现？

$$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} = \begin{bmatrix} 1 & -\frac{1-\cos \theta}{\sin \theta} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \sin \theta & 1 \end{bmatrix} \begin{bmatrix} 1 & -\frac{1-\cos \theta}{\sin \theta} \\ 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} = \begin{bmatrix} 1 & \frac{1-\cos \theta}{\sin \theta} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ -\sin \theta & 1 \end{bmatrix} \begin{bmatrix} 1 & \frac{1-\cos \theta}{\sin \theta} \\ 0 & 1 \end{bmatrix}$$

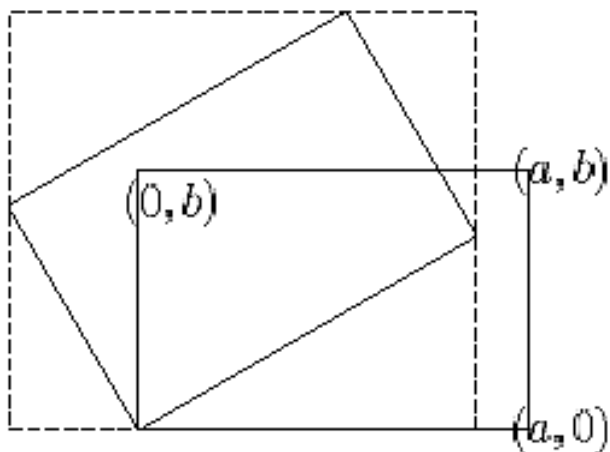


$$\text{注：} \tan\left(\frac{\theta}{2}\right) = \frac{1-\cos \theta}{\sin \theta}$$

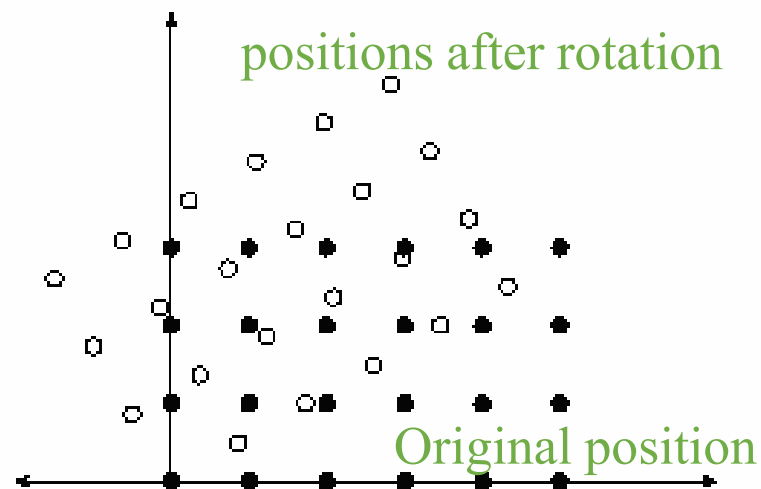
➤ Rotating a collection of points?

Not workable for images !

For images, rotated positions must remain in the image grid.



A rectangle surrounding a rotated image



Consider all integer-valued points (x', y') in the rectangle

A point will be in the image if, when rotated back, it lies within the original image limits.

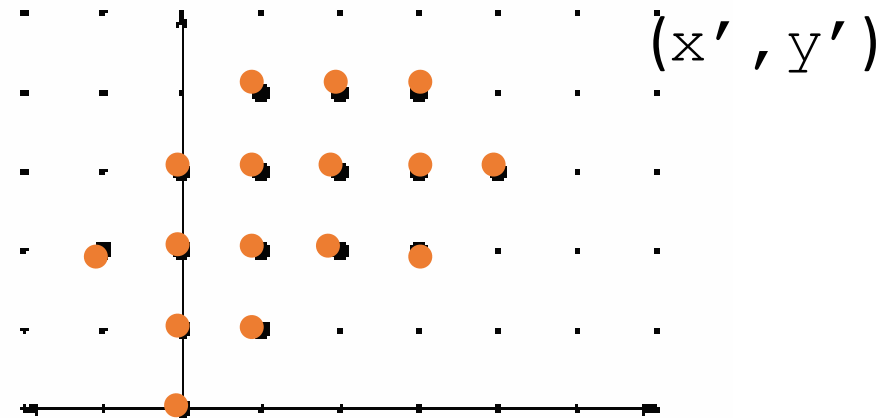
$$0 \leq x' \cos \theta + y' \sin \theta \leq a$$

$$0 \leq -x' \sin \theta + y' \cos \theta \leq b$$

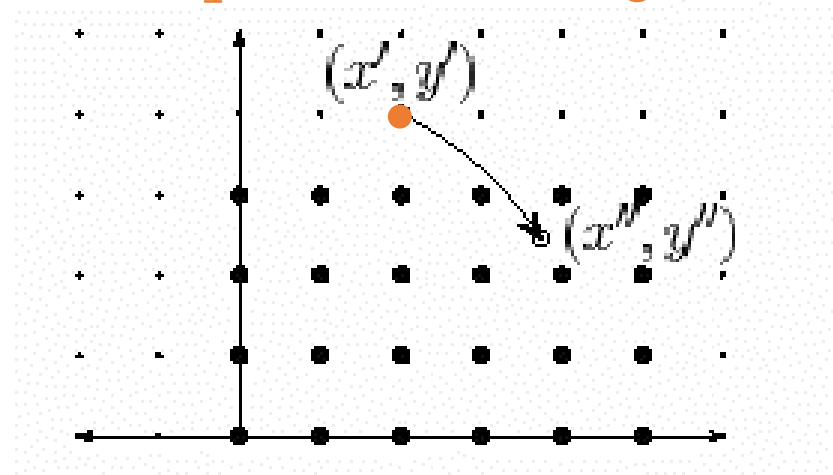
➤ 例: pixel positions of rotated image

Rotate (x', y') back into the original image to produce a point (x'', y'')

- The grey value at (x'', y'') can be found by interpolation.
- This value is the grey value for the pixel at (x', y') in the rotated image.



(x', y') in rotated image





➤ Image Rotation in Matlab: `imrotate`

Syntax: `B = imrotate(A,angle,method)`

- Rotates an image, using a specified interpolation method (default nearest neighbor) and rotation angle.
- The rotation angle is in degrees. For positive value, `imrotate` rotates the image counter-clockwise; For a negative value, it rotates the image clockwise.

The returned image `B` is, in general, larger than `A` to include the whole rotated image. `imrotate` sets invalid values on the periphery (外围) of `B` to 0.

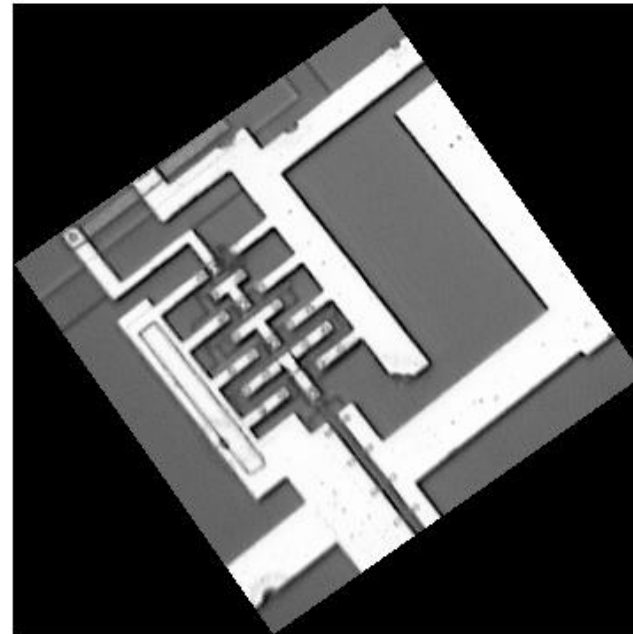
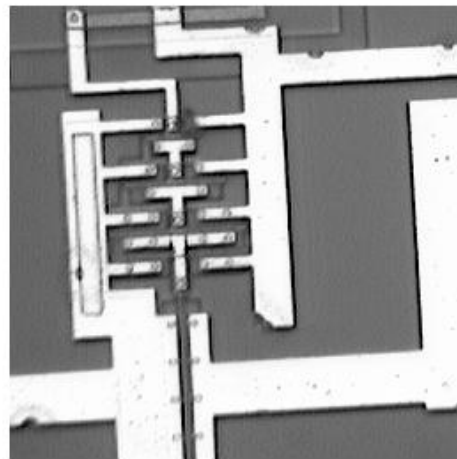
`B = imrotate(A,angle,method, 'crop')`

Rotates `A` through `angle` degrees and returns the central portion (same size as `A`).

➤ Image Rotation in Matlab: `imrotate`

例:

```
I = imread('ic.tif');  
J = imrotate(I,35,'bilinear');  
imshow(I)  
figure, imshow(J)
```



图像的旋转变换在深度学习领域的应用举例



数据扩充 (data augmentation)

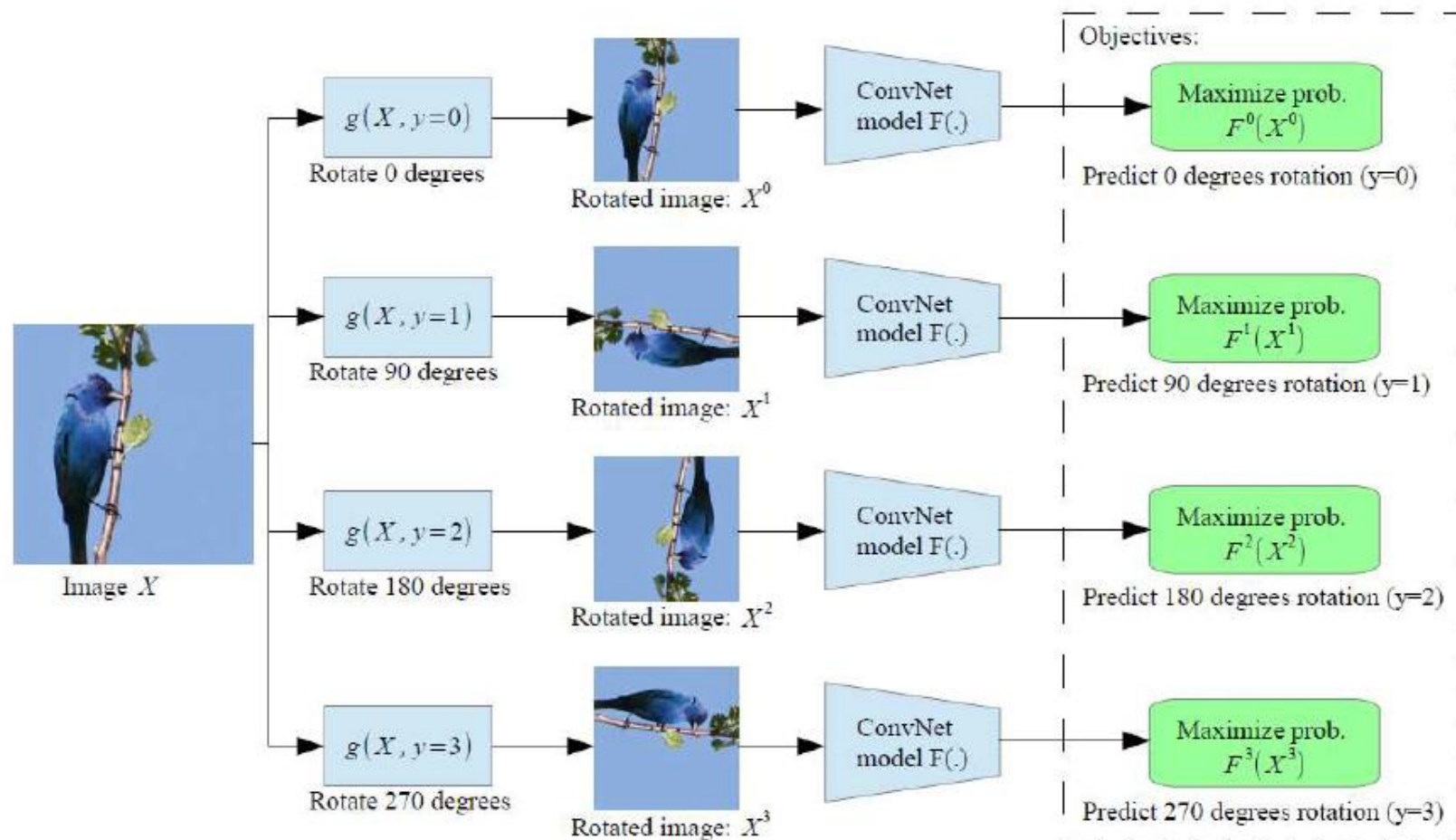
Which image has the correct rotation?



Unsupervised representation learning by predicting image rotations,
Spyros Gidaris, Praveer Singh, Nikos Komodakis, ICLR 2018

自监督学习 (Self-supervised learning)

图像的旋转变换在深度学习领域的应用举例



Unsupervised representation learning by predicting image rotations,
Spyros Gidaris, Praveer Singh, Nikos Komodakis, ICLR 2018

自监督学习 (Self-supervised learning)

仿射变换

- 仿射变换通过应用平移、旋转、缩放操作的线性组合，将变量（例如，位于输入图像中位置的像素强度值）映射到新变量（例如，在输出图像中）。
- 意义：**在一些成像系统中，图像会受到几何畸变的影响。对均匀失真的图像应用仿射变换可以校正一系列的透视失真。





- 仿射变换等价于平移、旋转和缩放的综合效应。
- 一般的仿射变换通常用齐次坐标表示，如下所示：

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = A \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} + B$$

- By defining only the B matrix, this transformation can carry out pure **translation**:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, B = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$



- Pure **rotation** uses the A matrix and is defined as (for positive angles being counter-clockwise rotations):

$$A = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}, B = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Similarly, pure **scaling** is:

$$A = \begin{bmatrix} a_{11} & 0 \\ 0 & a_{22} \end{bmatrix}, B = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Since the general affine transformation is defined by 6 constants, it is possible to define this transformation by specifying the new output image locations (x_2, y_2) of any three input image coordinate (x_1, y_1) pairs



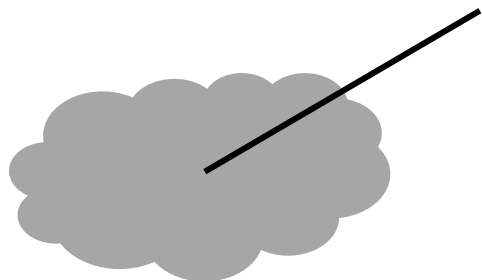
图像处理4：边缘检测 (edge detection)

- 物体的边缘以图像局部特征不连续的形式出现，也就是指图像局部亮度变化最显著的部分，例如：灰度值的突变，颜色的突变和纹理结构的突变等，同时物体的边缘也是不同区域的分界处。
- 图像边缘有方向和幅度两个特性，通常沿边缘走向的像素灰度变化平缓，垂直于边缘走向的像素灰度变化剧烈。
- 边缘检测的实质是采用某种算法来提取出图像中对象与背景间的交界线。图像灰度的变化情况可以用图像灰度分布的梯度来反映，因此我们可以用局部图像微分技术来获得**边缘检测算子**。
- 经典的边缘检测方法，是通过对原始图像中像素的某小邻域构造边缘检测算子来达到检测边缘这一目的（**使用差分近似图像函数导数的算子**）。

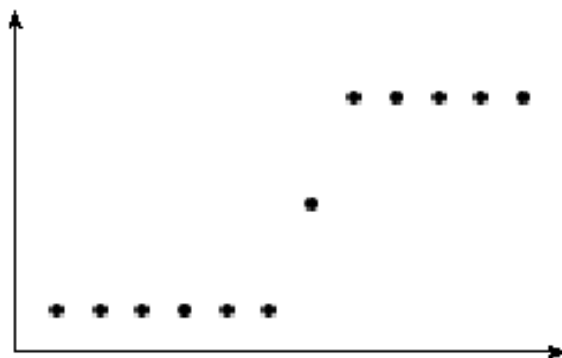
差分与边缘检测

在一个邻域中对像素的灰度平均会使图像中的细节变得模糊。差分产生相反的效果，即锐化图像。

边缘具有斜坡的一维形状，计算图像的导数会突出显示其位置。

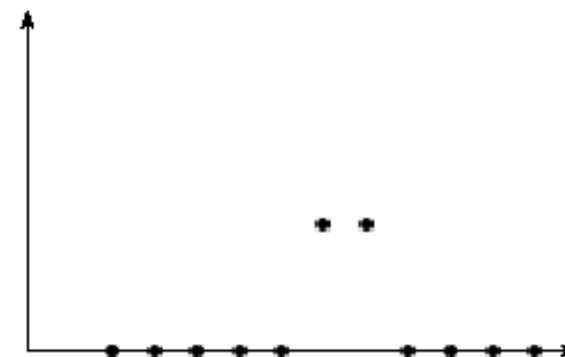


A line crossing an edge



Grey values across edges
(a ramp edge)

斜坡



Difference of the
Edge function

- The difference tends to enhance edges, and reduce other components.
- So we could difference image to get the edge effect



在对图像的操作，我们采用模板对原图像进行卷积运算，从而达到我们想要的效果。而获取一幅图像的梯度就转化为：**模板（Roberts、Prewitt、Sobel、Lapacian算子）对原图像进行卷积。**

- 在**一维**连续数集上有函数 $f(x)$ ，我们可以通过求导获得该函数在任一点的斜率，根据导数的定义有：

$$f'(x) = \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

- 在**二维**连续数集上有函数 $f(x,y)$ ，我们也可以通过求导获得该函数在 x 和 y 分量的偏导数，根据定义有：

$$\frac{\partial f(x, y)}{\partial x} = \frac{f(x + \Delta x, y) - f(x, y)}{\Delta x}$$

$$\frac{\partial f(x, y)}{\partial y} = \frac{f(x, y + \Delta y) - f(x, y)}{\Delta y}$$



对于图像来说，是一个二维的离散型数集，通过推广二维连续型求函数偏导的方法，来求得图像的偏导数，即在 (x,y) 处的最大变化率，也就是这里的梯度：

$$g_x = \frac{\partial f(x, y)}{\partial x} = f(x+1, y) - f(x, y)$$

$$g_y = \frac{\partial f(x, y)}{\partial y} = f(x, y+1) - f(x, y)$$

梯度是一个矢量，则 (x,y) 处的梯度表示为：

$$\nabla f \equiv \text{grad}(f) \equiv [g_x, g_y]^T = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]^T$$

其大小为：

$$M(x, y) = \text{mag}(\nabla f) = \sqrt{g_x^2 + g_y^2}$$

梯度和Roberts算子

平方和平方根需要大量的计算开销，所以使用绝对值来近似梯度幅值： $M(x, y) \approx |g_x| + |g_y|$

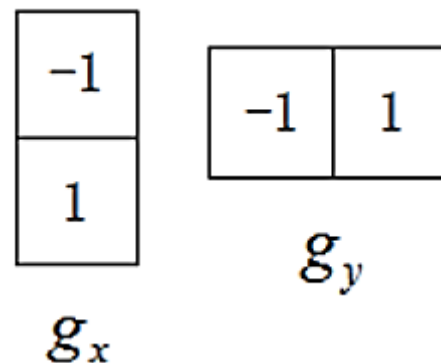
方向与 $\alpha(x, y)$ 正交： $\alpha(x, y) = \arctan \left[\frac{g_y}{g_x} \right]$

上右图是图像的垂直和水平梯度，但我们有时候也需要对角线方向的梯度，定义如下：

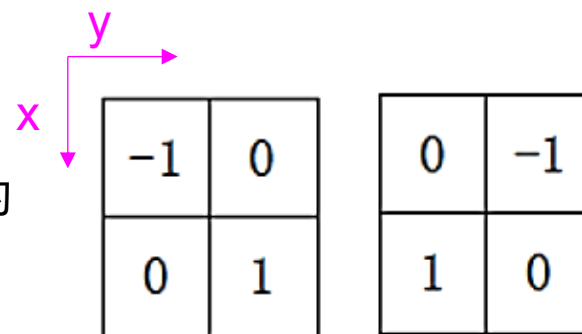
$$g_x = \frac{\partial f}{\partial x} = f(x+1, y+1) - f(x, y)$$

$$g_y = \frac{\partial f}{\partial y} = f(x+1, y) - f(x, y+1)$$

垂直和水平梯度的模板为：



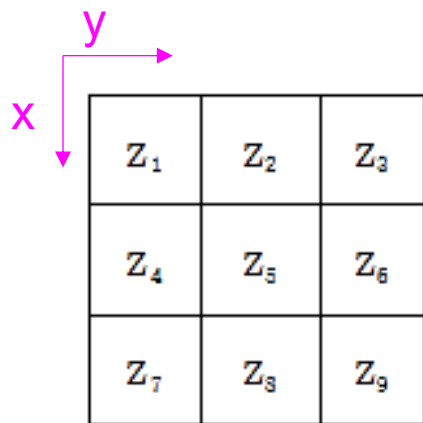
对角线梯度的模板为：



Roberts算子

- 2*2大小的模板概念简单，但对于用关于中心点对称的模板来计算边缘方向不是很有用，其最小模板大小为3*3。
- 3*3模板考虑了中心点对称数据的性质，并携带有关于边缘方向的更多信息。

Prewitt 算子



3*3 模板

定义水平、垂直和两对角线方向的梯度：

- 水平方向：
$$g_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$
- 垂直方向：
$$g_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$
- 对角线方向：
$$g'_x = (z_2 + z_3 + z_6) - (z_4 + z_7 + z_8)$$

$$g'_y = (z_6 + z_8 + z_9) - (z_1 + z_2 + z_4)$$

该定义下的算子称之为Prewitt算子：

-1	-1	-1
0	0	0
1	1	1

水平

-1	0	1
-1	0	1
-1	0	1

垂直

0	1	1
-1	0	1
-1	-1	0

对角线

-1	-1	0
-1	0	1
0	1	1



Sobel算子

Sobel算子是在Prewitt算子的基础上改进的，在中心系数上使用一个权值2，相比较Prewitt算子，Sobel模板能够较好的抑制（平滑）噪声。

计算公式为：

$$g_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$
$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

Sobel算子：

-1	-2	-1
0	0	0
1	2	1

水平

-1	0	1
-2	0	2
-1	0	1

垂直

0	1	2
-1	0	1
-2	-1	0

对角线

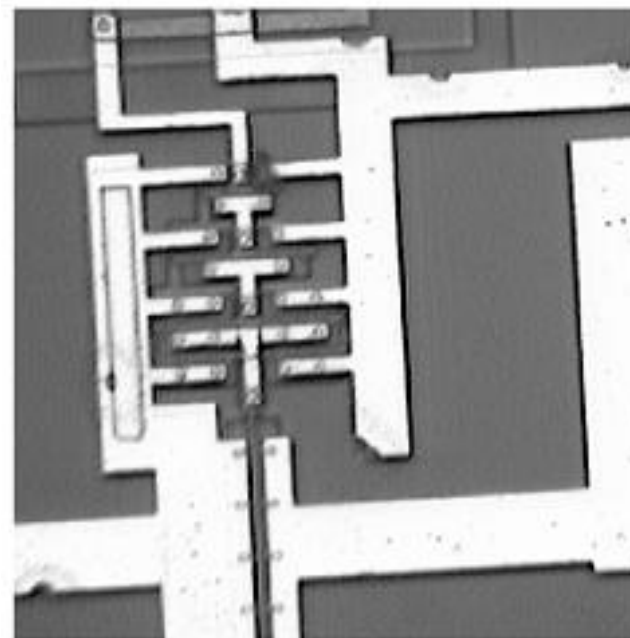
-2	-1	0
-1	0	1
0	1	2

例:

```
ic=imread('ic.tif');  
px=[-1 0 1;-1 0 1;-1 0 1];  
icx=filter2(px,ic);  
figure,imshow(icx/255)  
py=px';  
icy=filter2(py,ic);  
figure,imshow(icy/255)  
edge_p=sqrt(icx.^2+icy.^2);  
figure, imshow(edge_p/255)  
edge_t=im2bw(edge_p/255,0.3);
```

We can obtain edges by the Prewitt filters directly by using the command

```
edge_p=edge(ic,'prewitt');
```





Function `edge` find edges in intensity image.

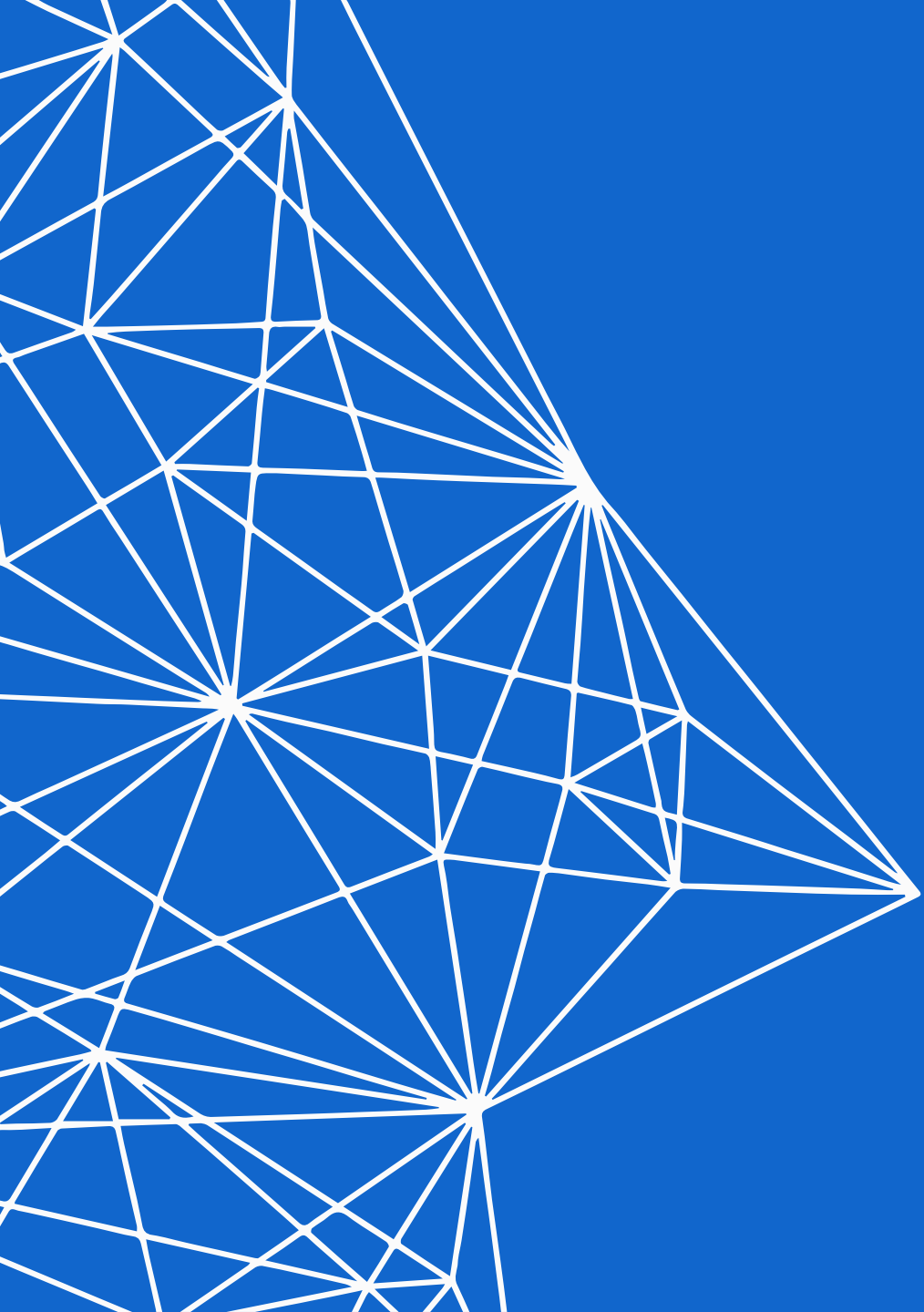
- `EDGE` takes an intensity image `I` as its input, and returns a binary image `BW` of the same size as `I`, with 1's where the function finds edges in `I` and 0's elsewhere.
- `EDGE` supports six different edge-finding methods:
 - The Sobel method
 - The Prewitt method
 - The Roberts method
 - The Laplacian of Gaussian method
 - The zero-cross method
 - The Canny method

Matlab edge function

例:

```
I=imread('Will.jpg');  
I=rgb2gray(I);  
BW1=edge(I,'sobel');  
BW2=edge(I,'prewitt');  
BW3=edge(I,'roberts');
```





THANKS!